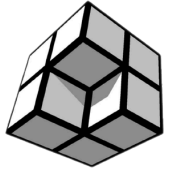


Robotics and Machine Intelligence



National Institute of Technology

Tiruchirappalli, Tamil Nadu - 620015

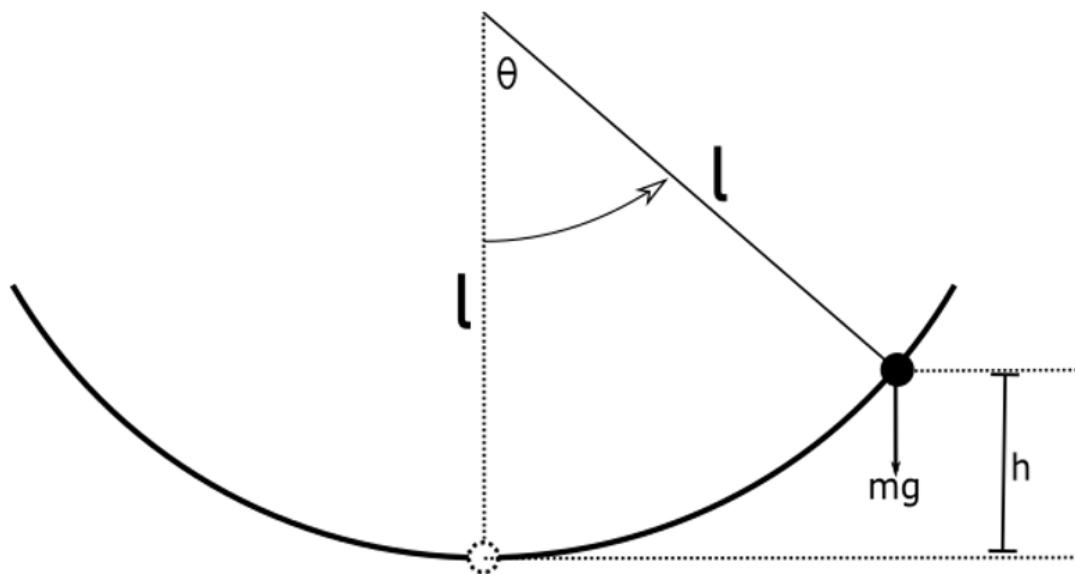
Balancing Builder Bot eYantra Team BB#2091

- Description: Balancing Builder Bot is a project by RMI Club at the National Institute of Technology, Tiruchirappalli. This project involves the simulation of a Two-wheel Self Balancing Robot with a gripper for controlled traversal of a given path crossing various obstacles and perform pick and place operations. The objective is to achieve a stable and efficient movement of the robot, ensuring proper pickup and drop of foam boxes while traversing a predefined path using control theory.
- Duration: Sep 2024 - Nov 2024
- Project link:
 - Drive link- [Balancing-builder](#)
- Status of Project: Completed
- Brief Outcomes: Throughout the project, the team has achieved the following outcomes:
 - Developed a mathematical model of an inverted pendulum as a basis for system understanding and balancing the bot.
 - Simulated the balancing of bot, path traversal and pick and place operations in CoppeliaSim using the mathematical model and implementing LQR control algorithm.

Project Report

Mathematical modelling of Inverted pendulum

The first part of the project is to develop a mathematical model and find the LQR gains for balancing the robot in CoppeliaSim. The mathematical model is developed in MATLAB.



To mathematically model the equation, we used the Euler Lagrangian method.

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{x}} \right) - \frac{\partial L}{\partial x} = 0$$

Where L = Kinetic Energy – Potential Energy

$$L = KE - PE$$

Let x and $\dot{x}$ be the state variables of this system (in our case, it is the angular position and angular velocity of the pendulum).

However, if there is an external torque input given, then the Lagrangian equation will be,

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = T$$

Where T is the external torque

Consequently, the equations of motion which are derived are

$$\begin{aligned}\dot{x}_1 &= x_2 \\ \dot{x}_2 &= \frac{-g}{l} \sin x_1 + \frac{1}{ml^2} T\end{aligned}$$

To analyse the system, state space equations were created. These are very important to create our LQR control system.

$$\begin{aligned}\text{State Equation} &\Rightarrow \dot{x}(t) = Ax(t) + Bu(t) \\ \text{Output Equation} &\Rightarrow y(t) = Cx(t) + Du(t)\end{aligned}$$

Here: -

$x(t)$ – State Vector

$u(t)$ – Input Vector

$y(t)$ – Output Vector

A – State Matrix

B - Input Matrix

C – Output Matrix

D – Feed forward Matrix

To find the A and B matrices, we use Jacobian.

$$A = \frac{\partial(\dot{x}_1, \dot{x}_2)}{\partial(x_1, x_2)}$$

$$B = \frac{\partial(\dot{x}_1, \dot{x}_2)}{\partial(u)}$$

- Calculating A for the system:

$$J_1 = \begin{bmatrix} \frac{\partial(x_2)}{\partial x_1} & \frac{\partial(x_2)}{\partial x_2} \\ \frac{\partial(-\frac{g}{l} \sin x_1 + \frac{1}{ml^2} T)}{\partial x_1} & \frac{\partial(-\frac{g}{l} \sin x_1 + \frac{1}{ml^2} T)}{\partial x_2} \end{bmatrix}$$

$$J_1 = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} \cos x_1 & 0 \end{bmatrix}$$

- Calculating B for the system:

$$J_2 = \begin{bmatrix} \frac{\partial f_1}{\partial u} \\ \frac{\partial f_2}{\partial u} \end{bmatrix}, \quad f_1 = \dot{x}_1, f_2 = \dot{x}_2$$

Since T is input, we replace T with u

$$J_2 = \begin{bmatrix} \frac{\partial(x_2)}{\partial u} \\ \frac{\partial(-\frac{g}{l} \sin x_1 + \frac{1}{ml^2} u)}{\partial u} \end{bmatrix}$$

$$J_2 = \begin{bmatrix} 0 \\ \frac{1}{ml^2} \end{bmatrix}$$

To check for stability, we need the equilibrium points. To find these, we take the conditions:

$$\dot{x}_1 = 0, \dot{x}_2 = 0$$

By solving this, we get $(x_1, x_2) \equiv (0, 0)$ and $(\pi, 0)$

The eigen values of A show if the equilibrium points have stable or unstable equilibrium.

$$A_{(0,0)} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{l} & 0 \end{bmatrix}, \quad A_{(\pi,0)} = \begin{bmatrix} 0 & 1 \\ \frac{g}{l} & 0 \end{bmatrix}$$

Therefore, the eigen values of each are:

- For (0,0) point:

$$\lambda = \pm \sqrt{\frac{g}{l}} i$$

- For $(\pi, 0)$ point :

$$\lambda = \pm \sqrt{\frac{g}{l}}$$

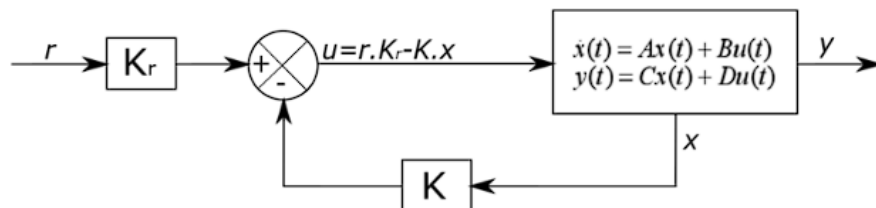
- If all the eigenvalues have negative real part, the system is **Stable** at the given Equilibrium point.
- If even one of the eigenvalues has positive real part, the system is **Unstable** at the given Equilibrium point.

At (0,0), it is marginally stable at $(\pi, 0)$, it is unstable.

Mathematical model and Controller Design for balancing bot

The state space representation is:

$$\begin{aligned}\dot{x}(t) &= Ax(t) + Bu(t) \\ y(t) &= Cx(t) + Du(t)\end{aligned}$$



Where: -

r – reference point

Kr – Gain by which reference is multiplied

x – State vector

y – Output vector

K – Gain by which input is multiplied

u – Input to system

For the control algorithm, we use LQR model which stands for Linear Quadratic Regulator. This helps us choose the optimum gain matrix K.

- We assume $u = -Kx$
- Cost function for LQR is

$$J = \int_0^{\infty} (x^T Q x + u^T R u)$$

Where Q and R are positive semi definite diagonal matrices.

- For our robot, the states are $x, \dot{x}, \theta, \dot{\theta}$.

- Hence, if our state vector is $X = (x, \dot{x}, \theta, \dot{\theta})^T$ and $K = (k_1, k_2, k_3, k_4)$, then

$$u = -KX = -(k_1x + k_2\dot{x} + k_3\theta + k_4\dot{\theta})$$

- The input u which is mapped between (-255 to 255). This is then fed to motor using analogWrite() function.
- The entire mathematical model and computations are done in MATLAB. The gain values are obtained from using lqr(A, B, Q, R) function.

Let m_{body} and m_{wheel} be the mass of body and wheel respectively.

Let r = radius of wheel

I_{wheel} = Moment of inertia of wheel about its axis

I_{body} = Moment of inertia of body about its centre of mass

L = Distance of COM from axis of wheel

$I_{\text{body_wheel}}$ = Moment of inertia of body about wheel axis

These are the important physical parameters of our system.

- $KE_{\text{Total}} = KE_{\text{Translation}} + KE_{\text{Rotation of body}} + KE_{\text{wheels}}$

$$KE_{\text{Total}} = \frac{1}{2} (m_{\text{body}} + 2m_{\text{wheel}}) \dot{x}^2 + \frac{1}{2} I_{\text{body_wheel}} \dot{\theta}^2 + 2 \left(\frac{1}{2} (m_{\text{wheel}}) \frac{\dot{x}^2}{r^2} \right)$$

- $PE = m_{\text{body}} g L \cos \theta$

Therefore, $L = KE_{\text{Total}} - PE$

The values of constants are given by Eyantra and also obtained from object properties in Coppeliasim.

Lagrangian property is find the state equations. Therefore,

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{x}} \right) - \frac{\partial L}{\partial x} = 0 \text{ and } \frac{d}{dt} \left(\frac{\partial L}{\partial \dot{\theta}} \right) - \frac{\partial L}{\partial \theta} = 0$$

From these equations, we can find X and $\dot{X}$.

Also, we know that

- $A = \text{jacobian}(\dot{X}, X)$
- $B = \text{jacobian}(\dot{X}, u)$

where `jacobian()` is a function in MATLAB.

Q and R are state cost and control cost matrices respectively. These penalise state error and control effort respectively.

- Q is a diagonal matrix where each element represents the priority in which the the states are corrected. Higher the value of element for a particular state, the more aggressive is the response to correct the deviation.
- R is a diagonal matrix which takes care of the control effort to the actuator. For a higher value of R, it gives a more energy efficient response rather than an aggressive response. In our case, we take $R = [1]$.

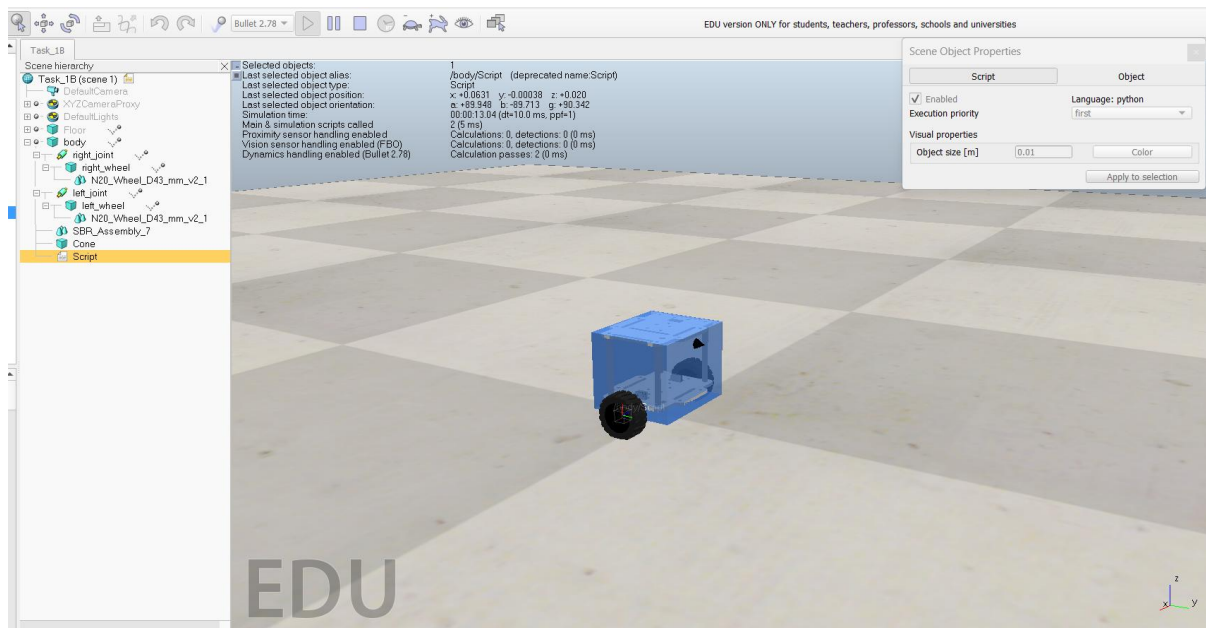
To find the LQR gains K, we use $K = \text{lqr}(A, B, Q, R)$. This function also gives solutions of the Algebraic Riccati equation, which is used to solve the cost function. We are only interested in the gain values.

Hence, we use the gain values to find our control input to the actuators.

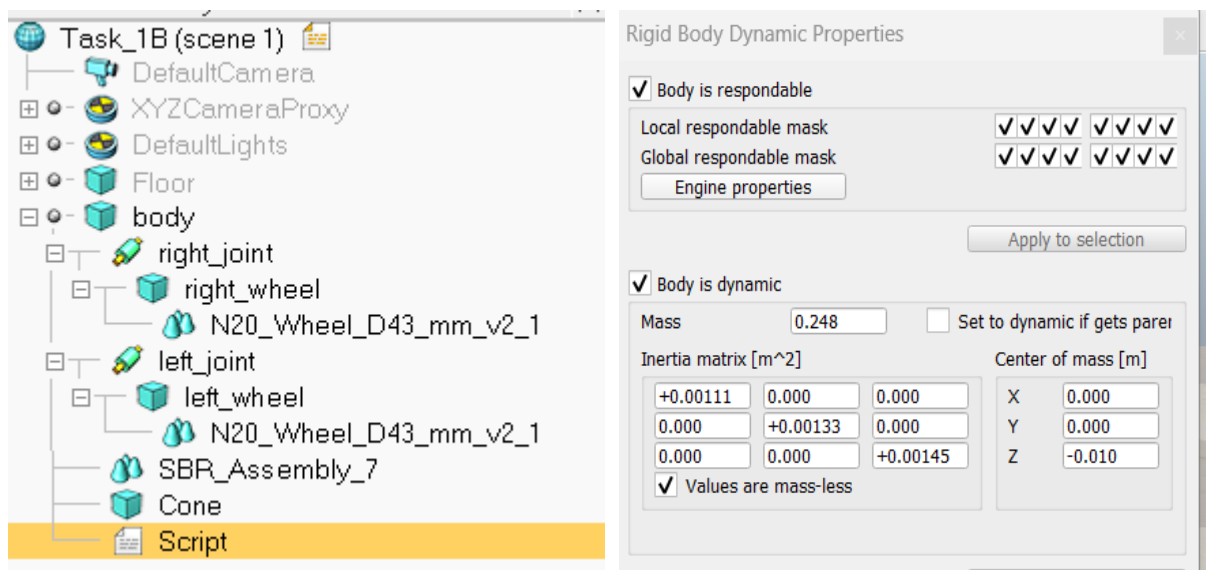
The code for mathematical model in MATLAB can be viewed from the GitHub repository.

Balancing the bot in CoppeliaSim

The first task is to balance the bot in CoppeliaSim and to traverse it along a path, while doing pick and place operations.



Self-Balancing Bot using LQR control



Hierarchy of the bot

Bot body properties

Python is used to implement the LQR model in CoppeliaSim. To get the states, we used the following functions: -

- `sim.getObjectPosition(self.body, sim.handle_world)[1]` – For x
- `sim.getObjectVelocity(self.body)[0][1]` – For $\dot{x}$
- `sim.getObjectOrientation(self.body, sim.handle_world)[0]` – For θ

- `sim.getObjectVelocity(self.body)[1][0]` – For $\dot{\theta}$

Where `self.body` and `sim.handle_world` are object handles for bot body and global frame.

Here, the indices 0,1,2 refer to parameters related to X, Y and Z axes of Global reference frame.

- `sim.getObjectVelocity(self.body)[0]` returns a list of $[V_x, V_y, V_z]$

where V is translational velocity on that axis.

- `sim.getObjectVelocity(self.body)[1]` returns a list of $[\omega_x, \omega_y, \omega_z]$

where ω is the angular velocity about that axis.

These are the states. So $X = [x, \dot{x}, \theta, \dot{\theta}]$. Therefore,

$$u = -(K_1 * X[0] + K_2 * X[1] + K_3 * X[2] + K_4 * X[3])$$

This is used for actuating the revolute joints using the functions,

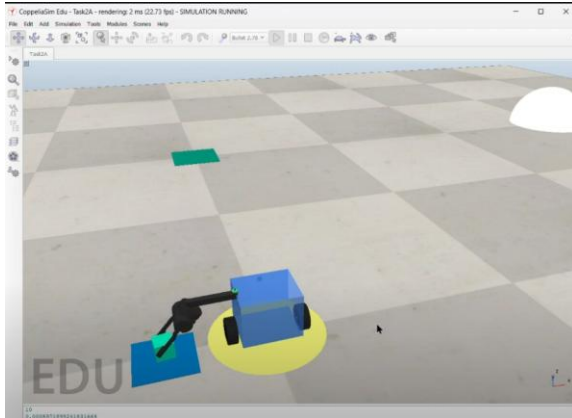
- `sim.setJointTargetVelocity(self.right_joint, u)`
- `sim.setvelocity(self.left_joint, u)`

where `self.right_joint` and `self.left_joint` are the object handles for the revolute joints of wheels. These are the joints which we need to actuate.

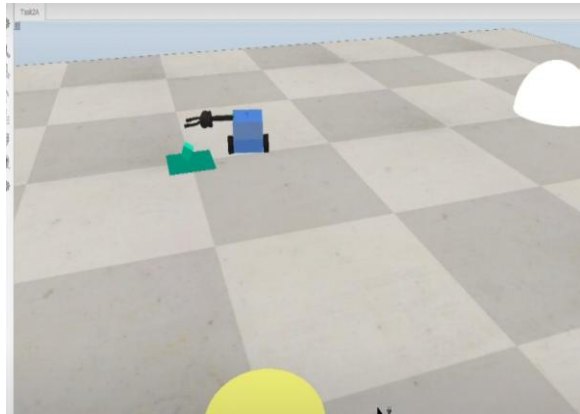
(Note: X is not vector in this case. It is a list)

Pick and place operation and path traversal

The next task is to conduct pick and place operations using the gripper attached to the bot on its side. This would vary the total mass of the bot and will add further challenges to the balance of bot. Fine tuning of gains is required for this.



Pickup of block by manipulator



Delivery of block by manipulator

The manipulator resembles a human arm, and it consists of one revolute joint at the top of the bot and one prismatic joint for the claw. We interface the keyboard commands with joint actuation, using `sim.GetSimulatorMessage()` and `sim.message_keypress` functions.

- For forward motion, we reduce the velocity of the joints by 10 (according to global reference frame)
- For backward motion, we increase the velocity of the joints by 10.
- For moving right / clockwise, we reduce the velocity of right joint by 2 and increase the left joint by 2
- For moving left / anti clockwise, we reduce the velocity of left joint by 2 and increase the right joint by 2.

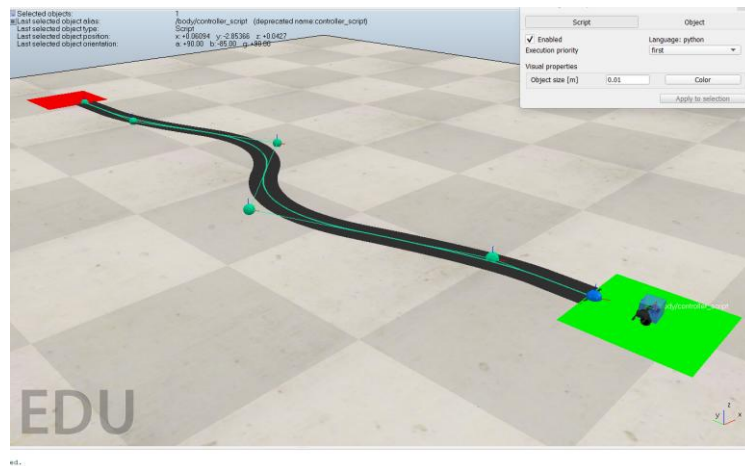
To ensure smooth and stable rotational motion, an additional adjustment is implemented for cases where the robot rotates beyond 45 degrees. When the bot's orientation crosses this threshold, the axis of rotation is dynamically changed. This is necessary to maintain consistent control over movement and prevent instability caused by large tilting angles.

- Before 45 degrees: The standard rotational control applies, where the left and right joints adjust their velocities to turn the bot.
- After 45 degrees: The rotation axis is recalibrated, meaning the reference frame for motion calculations is updated to account for the

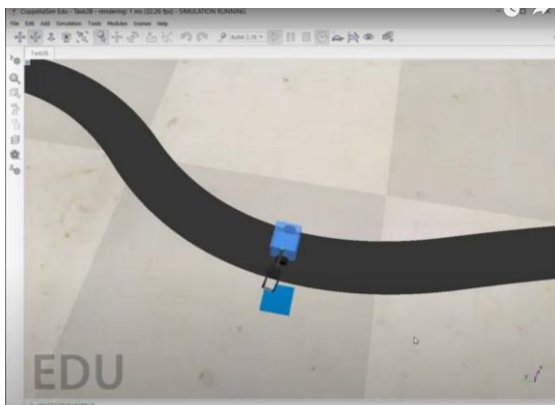
bot's new orientation. This adjustment ensures that the rotation remains smooth and does not become erratic due to exceeding a critical tilt angle.

By implementing this dynamic axis shift, the manipulator can handle a wider range of movements while maintaining precise control over its orientation, ultimately improving its manoeuvrability and stability.

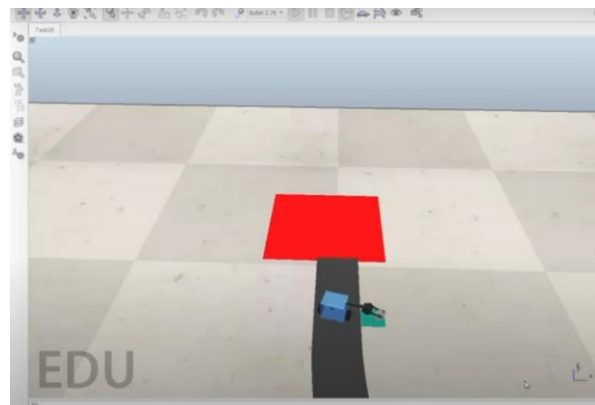
The next task is to traverse the bot over a particular path, while performing pick and place operations. The path is given by EYantra with a block which has to be picked at the pickup point and dropped at drop point using the gripper, while maintaining the self-balanced position and ensuring that the block does not touch the ground.



Path to be traversed



Bot picking up the block



Bot delivering the block

The full Coppeliasim code for balancing and traversing are available in the GitHub repository. The videos of simulation are available in the attached Google drive.